

TAIGA

EPIC: Chart Visualization & Dashboard Builder (Apache ECharts)

USER STORY 1: View and Manage Saved Charts

Points: 13

Description

As a user, I want to view all my previously saved charts so that I can reuse, edit, or organize them into dashboards.

Scenarios

- User opens app → sees list/grid of saved charts
 - User clicks a chart → opens in edit mode
 - User deletes a chart → it is removed from DB and UI
 - Charts persist across sessions
-

Tasks

◆ Task 1.1: Design Chart Metadata Model (5 pts)

Description:

Define a schema to persist chart configurations.

Details:

- Fields:
 - chart_id
 - chart_type (bar, line, etc.)
 - data_source_id
 - dimensions (array)

- measures (array)
 - axis_config
 - styling_config
 - validation_state
- Ensure extensibility for future charts (e.g., pie, heatmap)
-

◆ Task 1.2: Implement Chart Fetch API (3 pts)

Description:

Build backend API to fetch saved charts.

Scenario:

- GET /charts returns list with metadata + preview config
-

◆ Task 1.3: Build Frontend Chart Gallery with Virtualization (5 pts)

Description:

Efficiently render potentially large number of saved charts.

Details:

- Lazy rendering / virtualization
 - Thumbnail previews using ECharts config snapshots
 - Sorting (recent, name, type)
-

◆ Task 1.4: Implement Edit/Delete Chart Flow (3 pts)

Description:

Enable modification and deletion of charts.

Scenario:

- Edit → loads config into builder
 - Delete → soft delete with confirmation
-
-

USER STORY 2: Create New Chart via Configurator

Points: 21

Description

As a user, I want to create a new chart by selecting database, chart type, and fields so that I can visualize my data.

Scenarios

- User clicks "New Chart" → modal opens
 - User selects DB → tables → fields
 - User selects chart type → UI adapts dynamically
 - Invalid selections trigger validation errors
-

Tasks

◆ **Task 2.1: Build Data Source Explorer (5 pts)**

Description:

UI + backend integration to browse datasets.

Details:

- Fetch schemas dynamically
 - Handle large datasets (pagination/sample preview)
-

◆ **Task 2.2: Dynamic Chart Type Selection Engine (5 pts)**

Description:

Adapt UI based on chart type requirements.

Examples:

- Bar chart → categorical + numeric
 - Line chart → ordered dimension (time/sequence)
 - Box plot → distribution data
-

◆ Task 2.3: Field Mapping System (6 pts)

Description:

Allow users to assign:

- Dimensions (X-axis, grouping)
- Measures (Y-axis, aggregation)

Advanced Requirements:

- Support aggregation (SUM, AVG, COUNT)
 - Multi-series support
-

◆ Task 2.4: Query Builder Engine (5 pts)

Description:

Convert user selections into executable queries.

Details:

- SQL/NoSQL abstraction
 - Handle grouping, filtering, aggregation
 - Optimize for preview vs full dataset
-
-



USER STORY 3: Chart Rendering with Apache ECharts

Points: 21

Description

As a user, I want my selected data to render as interactive charts so that I can understand patterns visually.

Chart Types (Detailed)



Bar Chart

- Use: Compare categories
 - X-axis: categorical
 - Y-axis: numeric
 - Supports grouped & stacked bars
 - Validation:
 - Must have ≥ 1 dimension + ≥ 1 measure
-



Line Chart

- Use: Trends over time/sequence
 - X-axis: ordered (time or numeric index)
 - Supports multiple series
 - Validation:
 - X must be sortable
-



Box Plot

- Use: Distribution & outliers
 - Requires raw numeric data (not aggregated)
 - Displays median, quartiles, outliers
 - Validation:
 - Requires numeric dataset only
-



Additional Charts (recommended)

- Pie Chart → proportions (single dimension + measure)
 - Scatter Plot → correlation (2 numeric axes)
 - Heatmap → matrix-like density
-

Tasks

- ◆ **Task 3.1: ECharts Config Generator (8 pts)**

Description:

Transform normalized query output → ECharts option object.

♦ Task 3.2: Implement Chart Rendering Layer (5 pts)**Description:**

Reusable chart component.

Features:

- Responsive resizing
 - Theme support
 - Loading states
-

♦ Task 3.3: Multi-Series Handling Logic (4 pts)**Description:**

Support grouped/stacked charts dynamically.

♦ Task 3.4: Performance Optimization (4 pts)**Description:**

- Debounced rendering
 - Data sampling for large datasets
 - Progressive rendering
-
-

**USER STORY 4: Chart Customization**

Points: 13

Description

As a user, I want to customize chart appearance so that it fits my analysis needs.

Scenarios

- Change X/Y axis
 - Rename labels
 - Change colors/themes
 - Toggle legends
-

Tasks

◆ Task 4.1: Axis Configuration Engine (5 pts)

Description:

Allow dynamic reassignment of axes.

◆ Task 4.2: Styling Controls System (4 pts)

Description:

Expose ECharts styling:

- Colors
 - Grid
 - Tooltips
-

◆ Task 4.3: State Persistence for Customizations (4 pts)

Description:

Ensure all styling is saved and reloadable.



USER STORY 5: Chart Validation Engine

Points: 21

Description

As a user, I want the system to prevent invalid chart configurations so that I don't create meaningless visualizations.

Validation Rules (Core Logic)

Chart Type	Rules
Bar	≥1 categorical + ≥1 numeric
Line	X must be ordered
Box Plot	Only raw numeric values
Pie	Single dimension only
Scatter	Two numeric axes

Tasks

◆ **Task 5.1: Rule Engine Design (8 pts)**

Description:
Create a flexible validation framework.

◆ **Task 5.2: Schema-Aware Validation (5 pts)**

Description:
Use DB metadata to detect:

- data types
 - cardinality
 - null distributions
-

◆ **Task 5.3: Real-Time Validation Feedback (4 pts)**

Description:
Show errors instantly in UI.

◆ Task 5.4: Prevent Invalid Execution (4 pts)

Description:

Block query execution if invalid.



USER STORY 6: Save Charts for Dashboard Use

Points: 13

Description

As a user, I want to save charts so that I can reuse them in dashboards.

Scenarios

- User clicks save → chart persists
 - Chart appears in saved section
 - Can be reused in dashboards
-

Tasks

◆ Task 6.1: Save Chart API (5 pts)

Description:

POST endpoint to persist full chart config.

◆ Task 6.2: Versioning System (4 pts)

Description:

Track changes to charts.

◆ Task 6.3: Chart Serialization/Deserialization (4 pts)

Description:

Ensure configs are safely stored and reconstructed.



USER STORY 7: Dashboard Integration (Foundation)

Points: 8

Description

As a user, I want saved charts to be reusable in dashboards.

Tasks

- ◆ **Task 7.1: Dashboard-Ready Chart Format (4 pts)**

Description:

Standardize chart config for embedding.

- ◆ **Task 7.2: Chart Reference Linking (4 pts)**

Description:

Allow dashboards to reference charts without duplication.



EPIC: Dashboard Builder (Your Scope, Properly Structured)



USER STORY 1: View and Manage Saved Dashboards

Points: 13

Description

As a user, I want to access my saved dashboards so that I can continue working on them.

Scenarios

- Dashboard list loads with correct metadata
 - Selecting dashboard reconstructs full layout + charts
 - Deleted dashboards are removed without leaving dangling references
-

Tasks

◆ Task 1.1: Dashboard Retrieval & Reconstruction Pipeline (5 pts)

Description:

Implement backend + frontend flow to reconstruct dashboard from persisted state.

Key Work:

- Fetch layout config + chart IDs
 - Resolve chart configs from chart service
 - Merge into renderable state
 - Handle missing/deleted charts gracefully
-

◆ Task 1.2: Dashboard Preview Generation Strategy (4 pts)

Description:

Generate lightweight previews without rendering full charts.

Key Work:

- Create simplified layout snapshot
 - Avoid full ECharts instantiation
 - Cache previews for performance
-

◆ Task 1.3: Safe Deletion with Referential Integrity (4 pts)

Description:

Ensure deleting dashboards doesn't break chart references.

Key Work:

- Remove dashboard record
 - Preserve charts (since reusable)
 - Prevent orphaned layout states
-
-

USER STORY 2: Create New Dashboard with Chart Selection

Points: 21

Description

As a user, I want to create dashboards and populate them with charts.

Tasks

◆ **Task 2.1: Chart Selection → Dashboard State Mapping (6 pts)**

Description:

Convert selected charts into initial dashboard state.

Key Work:

- Assign unique layout IDs per chart
 - Initialize grid positions programmatically
 - Ensure no collisions in initial placement
-

◆ **Task 2.2: Dynamic Chart Injection into Grid (5 pts)**

Description:

Add charts into an existing dashboard without breaking layout.

Key Work:

- Detect free grid space
 - If no space → expand grid intelligently
 - Maintain layout consistency
-

◆ Task 2.3: Chart Removal with Layout Reflow (5 pts)

Description:

Remove a chart and reorganize layout.

Key Work:

- Remove layout node
 - Shift remaining elements to fill gaps
 - Avoid layout fragmentation
-

◆ Task 2.4: Chart Replacement Without Layout Loss (5 pts)

Description:

Replace chart while preserving layout slot.

Key Work:

- Swap chart reference only
 - Prevent unnecessary re-renders
 - Preserve dimensions/position
-
-

USER STORY 3: Drag, Resize, and Grid Layout Management

Points: 21

Description

As a user, I want to freely arrange charts using drag-and-drop and resizing.

Tasks

◆ Task 3.1: Controlled Grid State Management Layer (6 pts)

Description:

Wrap react-grid-layout with controlled state.

Key Work:

- Sync layout changes to React state
 - Normalize layout data structure
 - Prevent uncontrolled mutations
-

◆ Task 3.2: Resize Propagation to Charts (5 pts)

Description:

Ensure charts re-render correctly on resize.

Key Work:

- Detect dimension changes
 - Trigger ECharts resize API
 - Prevent unnecessary full re-renders
-

◆ Task 3.3: Drag Interaction Consistency Handling (5 pts)

Description:

Ensure smooth drag behavior.

Key Work:

- Prevent jitter during drag
 - Handle rapid position updates
 - Maintain stable layout state
-

◆ Task 3.4: Layout Persistence Trigger Strategy (5 pts)

Description:

Persist layout changes efficiently.

Key Work:

- Detect meaningful layout changes
 - Debounce updates
 - Avoid redundant saves
-
-

**USER STORY 4: Editable Dashboard Metadata**

Points: 8

Description

As a user, I want to edit dashboard properties like name.

Tasks**◆ Task 4.1: Inline Metadata Editing with Validation (3 pts)****Description:**

Allow editing with constraints.

Key Work:

- Prevent empty names
 - Handle duplicate names (if needed)
-

◆ Task 4.2: Partial Update API for Dashboard (3 pts)**Description:**

Update only metadata fields.

◆ Task 4.3: Sync UI State with Backend Updates (2 pts)

Description:

Ensure consistency after updates.



USER STORY 5: Annotation System on Charts

Points: 21

Description

As a user, I want to add annotations to charts.

Tasks

◆ Task 5.1: Annotation Positioning Relative to Chart Container (6 pts)

Description:

Store positions relative to chart dimensions.

Key Work:

- Normalize coordinates (0–1 scale)
 - Recalculate on resize
-

◆ Task 5.2: Annotation Interaction Model (6 pts)

Description:

Enable create/edit/delete.

Key Work:

- Drag annotations
 - Inline text editing
 - Multi-annotation support
-

◆ Task 5.3: Annotation Rendering Overlay System (5 pts)

Description:

Render annotations without interfering with charts.

◆ **Task 5.4: Annotation Persistence Integration (4 pts)**

Description:

Store annotations within dashboard state.



USER STORY 6: Dashboard Save & Consistency

Points: 13

Description

As a user, I want my dashboard changes to be reliably saved.

Tasks

◆ **Task 6.1: Full Dashboard State Serialization (5 pts)**

Description:

Convert dashboard into persistable format.

Includes:

- layout
 - chart references
 - annotations
-

◆ **Task 6.2: Backend Save API with Validation (4 pts)**

Description:

Ensure only valid dashboard states are stored.

◆ Task 6.3: Autosave with Change Detection (4 pts)

Description:

Save only when meaningful changes occur.
